

Age, SOPS, and Fly.io: Setup and Operation

Contents

1. Purpose and scope	4
2. The design in one diagram	5
3. Part 1 — age	8
3.1. What age is	8
3.2. Install	8
3.3. Generate a key pair	9
3.4. Protect the private key	9
3.5. The environment variable form	10
4. Part 2 — SOPS	11
4.1. What SOPS is	11
4.2. Install	11
4.3. Create the encrypted file	12
4.4. Editing the encrypted file	13
4.5. Adding a key	14
4.6. Rotating a value	14
4.7. Viewing a single value without opening the file	15
5. Part 3 — Fly.io Secrets	16
5.1. What Fly Secrets are	16
5.2. Install <code>flyctl</code>	16
5.3. Setting a secret	17
5.4. Listing secret names	17

5.5. Removing a secret	18
5.6. The bulk form	18
6. Part 4 — The <code>internal/credentials</code> package	19
6.1. What the package does	19
6.2. Fail-closed behaviour	19
6.3. Adding a new secret	20
6.4. Boot order	21
7. Part 5 — Day-to-day operations	22
7.1. First-time setup on a new machine	22
7.2. Rotating the age key	23
7.3. Rotating a secret without rotating the key	24
7.4. Checking what is deployed	24
7.5. Using the same key locally and in production	25
8. Part 6 — Troubleshooting	26
8.1. <code>no age key available</code>	26
8.2. <code>failed to decrypt secrets.enc.yaml: no identity matched any of the recipients</code>	26
8.3. <code>required secret JWT_SECRET is missing from secrets.enc.yaml</code>	26
8.4. <code>secret JWT_SECRET is 18 bytes; need at least 32 in production</code>	26
8.5. <code>JWT_SECRET</code> and <code>CSRF_SECRET</code> are the same value; they must differ	26
8.6. <code>credentials loaded count=2</code> but the file has six keys	27
8.7. The application starts but login fails with "invalid token"	27
8.8. <code>sops</code> refuses to open the key file	27
8.9. <code>sops</code> opens the file but the save fails	27
8.10. The Fly deployment restarts repeatedly after <code>fly secrets set</code>	27
9. Part 7 — What not to do	29
10. Part 8 — Files referenced by this document	30

11. Appendix — Command reference	31
12. Appendix — Environment variables read by <code>internal/credentials</code>	32
13. Appendix — The <code>secrets.enc.yaml</code> schema	33

patrick toca v0.8.3, October 7th, 2026

Creation Date: October 6th, 2026

Author: patrick toca

Email: pmjtoca@gmail.com

1. Purpose and scope

This document explains how **mywebapp** stores, encrypts, transports, and loads its secrets, and how those secrets reach a running process in production. It covers three tools and one small Go package:

- **age** — a file encryption tool. It generates a key pair and encrypts files to a recipient.
- **SOPS** — a tool that encrypts the **values** of a structured file (YAML, JSON, dotenv) while leaving the **keys** readable. It uses age (or another backend) as its cryptography.
- **Fly.io Secrets** — the platform's mechanism for injecting a small number of environment variables into a running machine, encrypted at rest and not readable back through the API.
- **internal/credentials** — the Go package in this repository that ties the three together. It reads the age private key from the environment, decrypts the SOPS file at boot, and sets each secret as an environment variable so the rest of the application reads secrets through **os.Getenv** exactly as it reads configuration.

By the end of this document you will be able to:

1. Install age and SOPS on a development machine.
2. Generate an age key pair and store the private key safely.
3. Create and edit an encrypted secrets file that is safe to commit.
4. Store the age private key as a Fly Secret.
5. Deploy an application that decrypts its own secrets at boot.
6. Rotate a secret without taking the application down.
7. Rotate the age key itself without losing access to the ciphertext.

2. The design in one diagram

The flow of a secret from your editor to the running process:

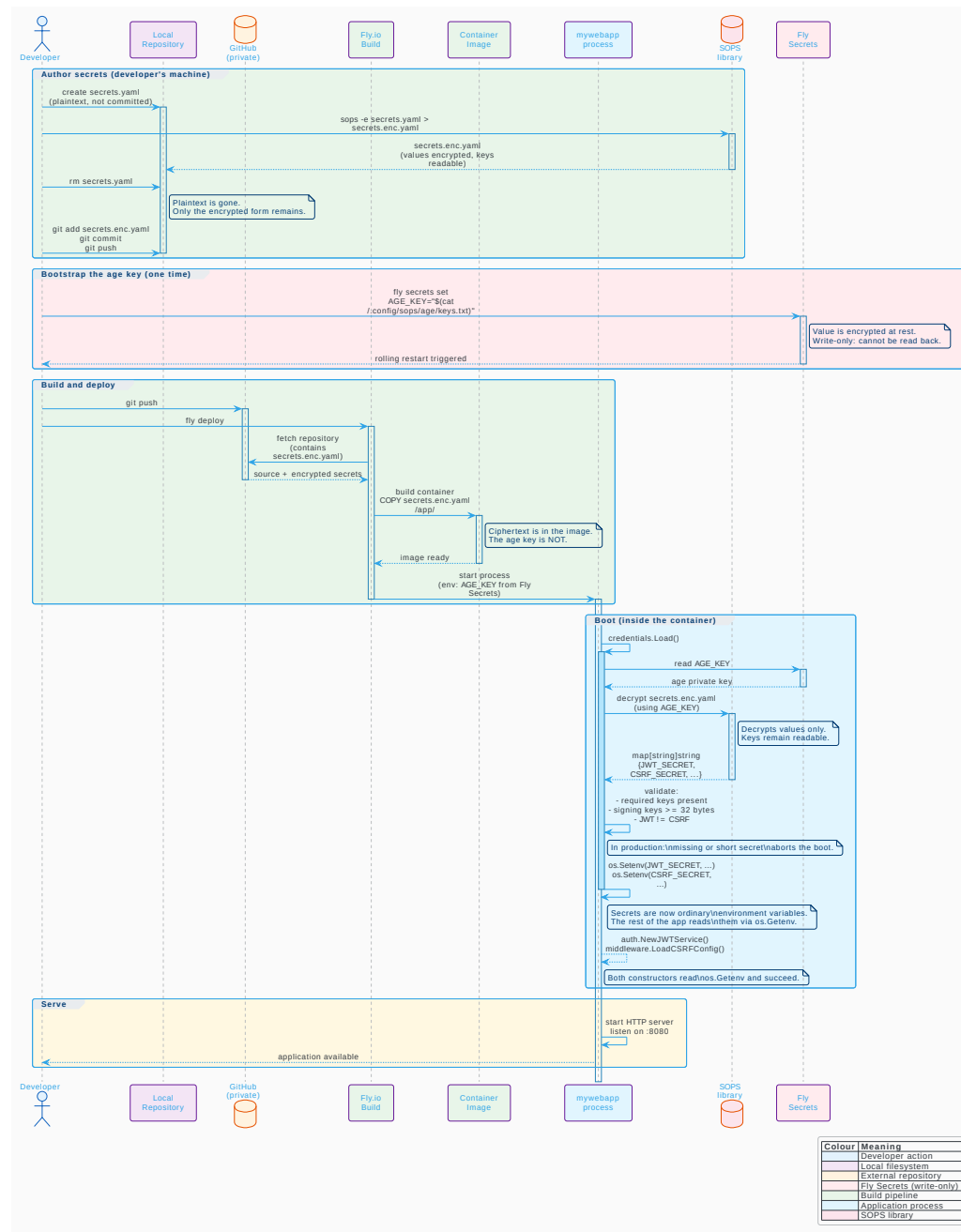


Figure 1. Request credentials flow with age, sops and fly in prod or in dev

The ciphertext is public. It lives in git, it is in the container image, and anyone with a copy of the repository can see it. The **only** thing that is secret is the age private key, which lives on your machine and in Fly's encrypted vault, and nowhere else.

Two properties follow from this design:

1. **git diff on the secrets file is meaningful.** Adding a key changes one line, and the line says which key. A pull request that rotates a secret is reviewable in the same way as a code change.
2. **The container is self-contained.** It carries the ciphertext and the loader. It needs one environment variable, `AGE_KEY`, to become functional. There is no sidecar, no external secret service, no network call to a secret store at boot.

3. Part 1 — age

3.1. What age is

age (pronounced "ah-gay", from the French **agé**) is a file encryption tool. You give it a public key (a "recipient") and a file; it produces an encrypted file that only the holder of the corresponding private key can decrypt. It is deliberately small: no configuration file, no keyring, no trust model beyond "you have the key or you do not."

SOPS uses age as one of its several cryptography backends. You do not interact with age directly once SOPS is configured, except to generate the key pair and to store the private key.

3.2. Install

On TUXEDO OS (and any Debian or Ubuntu derivative):

```
sudo apt update
sudo apt install age
```

Verify:

```
age --version
```

You should see 1.x or later. If apt does not carry a recent enough version, install the static binary from the release page:

```
# Replace vX.Y.Z with the latest release tag from
# https://github.com/FiloSottile/age/releases
curl -LO https://github.com/FiloSottile/age/releases/download/vX.Y.Z/age-vX.Y.Z-linux-amd64.tar.gz
tar xzf age-vX.Y.Z-linux-amd64.tar.gz
sudo install -m 0755 age/age /usr/local/bin/age
```

```
sudo install -m 0755 age/age-keygen /usr/local/bin/age-keygen
```

3.3. Generate a key pair

```
mkdir -p ~/.config/sops/age  
age-keygen -o ~/.config/sops/age/keys.txt
```

The command prints the **public key** to stderr. It looks like:

```
# created: 2026-10-06T10:00:00Z  
# public key: age1qz4m6k9...  
AGE-SECRET-KEY-1QZ4M6K9...
```

The first line of output is a comment. The second line is the public key; write it down. The file `~/.config/sops/age/keys.txt` contains the private key. The format is two lines: a comment with the public key, then the private key itself.



Lose the private key and you lose the ability to decrypt every file that was encrypted to it. There is no recovery mechanism. There is no "reset password" link. If the key is gone, the ciphertext in your git history is permanently unreadable.

Back the file up. Put a copy in a password manager, on an encrypted USB stick, or somewhere else that is not the machine the file lives on.

3.4. Protect the private key

```
chmod 600 ~/.config/sops/age/keys.txt
```

The `age` and `sops` tools refuse to read a key file with permissions looser than this. The `chmod` is not optional.

3.5. The environment variable form

Some tools, including the SOPS library and the `internal/credentials` package, accept the private key as an environment variable rather than a file. The variable name is `AGE_KEY`. This is the form Fly.io uses:

```
export AGE_KEY="$(cat ~/.config/sops/age/keys.txt)"
```

`internal/credentials` reads `AGE_KEY` first, then falls back to `SOPS_AGE_KEY_FILE`, then to the default path `~/.config/sops/age/keys.txt`. In development the file already exists, so you do not need to set `AGE_KEY`. In a container the file does not exist, so you do.

4. Part 2 — SOPS

4.1. What SOPS is

SOPS ("Secrets OPerationS") encrypts the **values** in a structured file while leaving the **keys** readable. A SOPS-encrypted YAML file looks like this:

```
DB_PASSWORD: ENC[AES256_GCM,data:aBcD...,iv:...,tag:...,type:str]
JWT_SECRET: ENC[AES256_GCM,data:eFgH...,iv:...,tag:...,type:str]
CSRF_SECRET: ENC[AES256_GCM,data:iJkL...,iv:...,tag:...,type:str]
sops:
  age:
    - recipient: age1qz4m6k9...
      enc: |
        -----BEGIN AGE ENCRYPTED FILE-----
        ...
    lastmodified: "2026-10-06T10:00:00Z"
    mac: ENC[AES256_GCM,data:...,type:str]
    version: 3.8.1
```

Two properties follow from this:

1. **git diff is meaningful.** Adding a key changes one line, and the line says which key. You can review a secrets change in a pull request the same way you review a code change.
2. **You can add a key without decrypting the whole file.** The keys are readable; only the values need the age key.

4.2. Install

SOPS is not in the Debian or Ubuntu repositories. Install the binary from the release page:

```
# Replace vX.Y.Z with the latest release tag from
# https://github.com/getsops/sops/releases
curl -LO https://github.com/getsops/sops/releases/download/vX.Y.Z/sops-vX.Y.Z.linux.amd64
sudo install -m 0755 sops-vX.Y.Z.linux.amd64 /usr/local/bin/sops
```

Verify:

```
sops --version
```

If you prefer to build from source:

```
go install github.com/getsops/sops/v3/cmd/sops@latest
```

The binary lands in `$(go env GOPATH)/bin/sops`. Add that directory to your `PATH` if it is not already there.

4.3. Create the encrypted file

Create the plaintext form first. The example below is what `mywebapp` expects; replace the placeholder values with real ones.

```
cat > secrets.yaml <<'EOF'
DB_PASSWORD: REPLACE_ME
TEST_DB_PASSWORD: REPLACE_ME
JWT_SECRET: REPLACE_ME
CSRF_SECRET: REPLACE_ME
R2_ACCESS_KEY_ID: REPLACE_ME
R2_SECRET_ACCESS_KEY: REPLACE_ME
EOF
```

Encrypt it to a file, then delete the plaintext:

```
sops --encrypt secrets.yaml > secrets.enc.yaml  
rm secrets.yaml
```

Verify that decryption works:

```
sops --decrypt secrets.enc.yaml
```

You should see the same YAML you wrote, with the values in plaintext.



The `.gitignore` in this repository already excludes `secrets.yaml`, `secrets.yml`, and `secrets.env`. The plaintext file cannot be committed by accident; the pattern is there as a guard, not as the only defence.

`secrets.enc.yaml` is not ignored. It is the ciphertext, it is safe to commit, and the review benefit described above depends on it being in the repository.

4.4. Editing the encrypted file

Do not decrypt to a file and re-encrypt. That leaves the plaintext on disk and creates a window where it can be read by another process or committed by a `git add ..`

Instead, edit the encrypted file directly:

```
sops secrets.enc.yaml
```

`SOPS` decrypts the file to a temporary location, opens `$EDITOR` on it, and on save re-encrypts the result back to `secrets.enc.yaml`. The plaintext is only ever in the editor's memory and in the temp file, and the temp file is deleted when the editor exits.

Set `$EDITOR` if it is not already set:

```
export EDITOR=nvim # or vim, nano, emacs, code --wait, ...
```

4.5. Adding a key

Open the file with `sops secrets.enc.yaml`, add a line in the editor, and save. For example, to add a new secret called `SMTP_PASSWORD`:

```
SMTP_PASSWORD: hunter2
```

Save and exit. SOPS re-encrypts the file. The new key is present, its value is encrypted.

If the application expects the new key, add it to the `knownSecrets` list in `internal/credentials/credentials.go` so the loader will export it into the environment. A key that is in the SOPS file but not in that list is present in the decrypted map but not set as an environment variable.

4.6. Rotating a value

Open the file, replace the value, save. Then commit and redeploy:

```
sops secrets.enc.yaml
git add secrets.enc.yaml
git commit -m "rotate JWT secret"
git push
fly deploy
```

A JWT secret rotation invalidates every existing access and refresh token. Users must log in again. That is the correct behaviour; a rotated secret that did not invalidate tokens would not be a rotation.

A CSRF secret rotation invalidates every issued CSRF cookie. Users get a fresh one on their next GET. No user-visible effect.

A database password rotation requires the new password to be set on the PostgreSQL role **before** the deployment rolls out. The rolling restart

replaces machines one at a time; during the overlap, old machines still hold the old password. Coordinating the change with the role update avoids an authentication failure on the machines that start first.

4.7. Viewing a single value without opening the file

For a quick check, without leaving a decrypted copy anywhere:

```
sops --extract '["JWT_SECRET"]' --decrypt secrets.enc.yaml
```

The output is the value only. This is useful when you want to paste the value into a shell command without opening an editor.

5. Part 3 — Fly.io Secrets

5.1. What Fly Secrets are

Fly.io has a mechanism for injecting environment variables into a running machine. It is encrypted at rest, and the values are **write-only**: once set, neither `flyctl` nor the Fly API can read them back. The API servers can encrypt but not decrypt. This is a deliberate design choice and it is why Fly Secrets are suitable for the age private key.

The trade-off: you cannot audit the value of a Fly Secret through the platform. If you forget what you set, you overwrite it. There is no "show me the current value" command. For a bootstrap secret like `AGE_KEY`, this is fine: the value is on your machine, and if you lose it you re-upload it.

5.2. Install `flyctl`

```
curl -L https://fly.io/install.sh | sh
```

The installer places `flyctl` in `~/.fly/bin`. Add it to your `PATH`:

```
export FLYCTL_INSTALL="$HOME/.fly"  
export PATH="$FLYCTL_INSTALL/bin:$PATH"
```

Add those two lines to your shell's startup file (`~/.zshrc` or `~/.bashrc`) to make them permanent.

Authenticate:

```
fly auth login
```

5.3. Setting a secret

```
fly secrets set AGE_KEY="$(cat ~/.config/sops/age/keys.txt)" --app mywebapp
```

The `--app` flag is optional if `fly.toml` names the application, which it does in this repository. The command:

1. Sends the value over TLS to Fly's API.
2. Stores it encrypted.
3. Triggers a rolling restart of the application's machines so the new environment variable is visible to the next boot.

The restart is automatic and unavoidable. Setting a secret is a deploy in miniature: the machines are replaced one at a time, with the new value visible to the replacements and not to the originals.

The command above uses `cat`, which includes the comment lines in the key file. The comment lines start with `and internal/credentials` handles them, but the `grep -v '^'` form used elsewhere in this document is the cleaner choice:

```
fly secrets set AGE_KEY="$(grep -v '^#' ~/.config/sops/age/keys.txt)" --app mywebapp
```

5.4. Listing secret names

```
fly secrets list --app mywebapp
```

The output shows the **names** and a digest (a short hash) of each secret, not the values. The digest is stable for a given value, so you can use it to confirm that a rotation took effect without reading the value back.

5.5. Removing a secret

```
fly secrets unset AGE_KEY --app mywebapp
```

Removing a secret also triggers a rolling restart. The next boot will fail with "no age key available" if the application still needs it, which is the fail-closed behaviour `internal/credentials` is designed for.

5.6. The bulk form

If you have multiple secrets to set at once, `fly secrets import` reads them from stdin in dotenv format:

```
fly secrets import --app mywebapp < .secrets.env
```

For `mywebapp`, only `AGE_KEY` needs to be set this way. Everything else lives in `secrets.enc.yaml`, which is decrypted at boot. Adding the other secrets to Fly's vault would create a second source of truth and defeat the point of the design.

6. Part 4 — The `internal/credentials` package

6.1. What the package does

The package has one exported function, `Load()`, and one responsibility: to make the application's secrets visible in the process environment before any service is constructed.

```
func Load() error
```

`Load` performs four steps in order:

1. Ensures the age private key is available to the SOPS library. It reads `AGE_KEY` from the environment and sets `SOPS_AGE_KEY` from it. If `AGE_KEY` is absent, it checks `SOPS_AGE_KEY_FILE` and the default path. If no key is found, it returns an error.
2. Decrypts `secrets.enc.yaml` using the SOPS library.
3. Validates that the secrets the application requires are present and, for signing keys, at least 32 bytes long.
4. Exports each secret in the `knownSecrets` list into the process environment with `os.Setenv`.

The rest of the application does not know about SOPS, age, or Fly. It reads `JWT_SECRET`, `CSRF_SECRET`, and `DB_PASSWORD` through `os.Getenv`, the same way it reads `DB_HOST` and `PORT`. The distinction between configuration and secrets is in where the values come from, not in how the code consumes them.

6.2. Fail-closed behaviour

In production and staging (`APP_ENV` set to `production` or `staging`), `Load` returns an error and `main` exits in any of these cases:

- `secrets.enc.yaml` is missing.
- No age key is available.
- `JWT_SECRET` or `CSRF_SECRET` is missing from the file.

- `JWT_SECRET` or `CSRF_SECRET` is shorter than 32 bytes.
- `JWT_SECRET` and `CSRF_SECRET` are the same value.

In development (`APP_ENV` unset, or set to `development`), the same conditions produce warnings instead. The process starts. The consumer's own fallback applies: `auth.NewJWTService` and `middleware.LoadCSRFConfig` each generate a random per-process value when the corresponding variable is empty. That is what makes `go run` work without ceremony, and the warning is what makes the missing secret visible.

The asymmetry is deliberate. In production, a missing secret is a deployment error and the correct response is to refuse to serve. In development, a missing secret is an inconvenience and the correct response is to keep the developer moving while making the gap visible in the log.

6.3. Adding a new secret

Three edits, always in the same three places:

1. Add the key and its value to `secrets.enc.yaml`. Use `sops secrets.enc.yaml` to edit the file in place.
2. Add the key's name to the `knownSecrets` slice in `internal/credentials/credentials.go`.
3. Add a read of the key where the application needs it: `os.Getenv("MY_NEW_SECRET")`.

The `knownSecrets` list is a closed set. A key that is in the encrypted file but not in the list is present in the decrypted map but not exported as an environment variable. This is deliberate: it means a typo in the encrypted file does not silently become a live environment variable.

The loader logs a single line at boot:

```
INFO credentials loaded count=6
```

`count` is the number of secrets exported into the environment. If it is lower than the number of keys in `secrets.enc.yaml`, at least one key is not in the `knownSecrets` list and is being silently ignored. The log line is the only signal that the two have diverged; treat a change in `count` as a reason to inspect the file.

6.4. Boot order

`Load` must run before any service that reads a secret is constructed. In `cmd/api/main.go`, this means before `auth.NewJWTService`, `middleware.LoadCSRFConfig`, and `db.NewPool`. The call is placed at the top of `main`, immediately after the logging setup, so the environment is populated before anything else reads it.

7. Part 5 — Day-to-day operations

7.1. First-time setup on a new machine

```
# 1. Install age and SOPS (see Parts 1 and 2).

# 2. Restore the age private key. If you backed it up in a password
#    manager, retrieve it and write it to the default location.
mkdir -p ~/.config/sops/age
$EDITOR ~/.config/sops/age/keys.txt
chmod 600 ~/.config/sops/age/keys.txt

# 3. Clone the repository.
git clone git@github.com:you/mywebapp.git
cd mywebapp

# 4. Verify the SOPS file decrypts.
sops --decrypt secrets.enc.yaml > /dev/null && echo "ok"

# 5. Create the local configuration.
cp .env.example .env
$EDITOR .env

# 6. Run.
go run ./cmd/api
```

If step 4 fails, the age key on this machine does not match the one the file was encrypted to. Recover the correct key or re-encrypt the file to the new machine's public key. There is no way to decrypt a file without the private key it was encrypted to.

7.2. Rotating the age key

Rotating the age key means re-encrypting the SOPS file to a new recipient and changing the `AGE_KEY` secret on Fly. Existing ciphertext encrypted to the old key cannot be decrypted with the new one, so the re-encryption must happen before the old key is discarded.

```
# 1. Generate a new key pair.
age-keygen -o ~/.config/sops/age/keys-new.txt
# Note the new public key: age1...

# 2. Re-encrypt the SOPS file to the new recipient, keeping the old one
#    as a fallback in case the new key is lost.
sops --add-age age1newpublickey... secrets.enc.yaml

# 3. Verify the new key can decrypt.
SOPS_AGE_KEY_FILE=~/.config/sops/age/keys-new.txt \
    sops --decrypt secrets.enc.yaml > /dev/null

# 4. Update Fly.
fly secrets set AGE_KEY="$(grep -v '^#' ~/.config/sops/age/keys-new.txt)"

# 5. Deploy.
git add secrets.enc.yaml
git commit -m "re-encrypt secrets to new age key"
git push
fly deploy

# 6. Once the deployment is confirmed healthy, remove the old recipient.
sops --rm-age age1oldpublickey... secrets.enc.yaml
git add secrets.enc.yaml
git commit -m "remove old age recipient"
fly deploy

# 7. Replace the local key file.
```

```
mv ~/.config/sops/age/keys-new.txt ~/.config/sops/age/keys.txt
chmod 600 ~/.config/sops/age/keys.txt
```

Step 2 keeps both recipients. If the new key is lost between step 2 and step 6, the old key still decrypts the file. Step 6 removes the old recipient once the new one is confirmed working.

If you have other machines with the old key, they lose access at step 6. Distribute the new key to them before running step 6, or accept that they will need to be re-provisioned.

7.3. Rotating a secret without rotating the key

The common case. Open the SOPS file, replace the value, commit, redeploy.

```
sops secrets.enc.yaml
git add secrets.enc.yaml
git commit -m "rotate JWT_SECRET"
git push
fly deploy
```

A JWT secret rotation invalidates every existing access and refresh token. Users must log in again. That is the correct behaviour; a rotated secret that did not invalidate tokens would not be a rotation.

A CSRF secret rotation invalidates every issued CSRF cookie. Users get a fresh one on their next GET. No user-visible effect.

7.4. Checking what is deployed

```
# Which age recipients can decrypt the committed file?
sops --extract '["sops"]["age"]' --decrypt-keys secrets.enc.yaml

# What is the digest of the current AGE_KEY on Fly?
```

```
fly secrets list --app mywebapp

# What does the application see at boot?
fly logs --app mywebapp | grep -i credential
```

The application logs a line at boot:

```
INFO credentials loaded count=6
```

`count` is the number of secrets exported into the environment. If it is lower than the number of keys in `secrets.enc.yaml`, at least one key is not in the `knownSecrets` list and is being silently ignored.

7.5. Using the same key locally and in production

The local key file and the Fly Secret hold the same value. That is intentional: the container that runs in production is built from the same repository as the one that runs locally, and both need to decrypt the same `secrets.enc.yaml`. If you want production to use different values from development, the correct approach is a second encrypted file (`secrets.prod.enc.yaml`) and a second recipient, not a second `AGE_KEY`.

8. Part 6 — Troubleshooting

8.1. no age key available

The error from `credentials.Load`. Causes, in order of likelihood:

1. `AGE_KEY` is unset on Fly. Check `fly secrets list`.
2. The age key file at `~/.config/sops/age/keys.txt` is missing or has permissions looser than `0600`.
3. `SOPS_AGE_KEY_FILE` is set to a path that does not exist.
4. In a container, `AGE_KEY` was not passed. Check that the env file is being read, or that `-e AGE_KEY=...` is on the command line.

8.2. failed to decrypt secrets.enc.yaml: no identity matched any of the recipients

The age key on this machine is not the one the file was encrypted to. Either you are on a machine with the wrong key, or the file was re-encrypted after the key was generated. Retrieve the correct key, or re-encrypt the file to the current machine's public key.

8.3. required secret JWT_SECRET is missing from secrets.enc.yaml

The key is absent from the file. Add it with `sops secrets.enc.yaml`, or, if this is development, accept the warning and let the fallback apply.

8.4. secret JWT_SECRET is 18 bytes; need at least 32 in production

The value is too short. Regenerate with `openssl rand -base64 48` and update the file.

8.5. JWT_SECRET and CSRF_SECRET are the same value; they must differ

Both keys hold the same string. They serve different purposes and must be different. Regenerate one of them.

8.6. `credentials` loaded `count=2` but the file has six keys

Only two keys are in the `knownSecrets` list. Add the others. The count is the number of keys the loader exported, not the number the file contains.

8.7. The application starts but login fails with "invalid token"

The `JWT_SECRET` the running process sees is different from the one that issued the token. Either the secret was rotated between the token being issued and the request, or the deployment is running with a stale environment. Check `fly logs` for the boot sequence and confirm `credentials.Load` ran before `auth.NewJWTService`.

8.8. `sops` refuses to open the key file

The permissions on `~/.config/sops/age/keys.txt` are looser than `0600`. Fix with:

```
chmod 600 ~/.config/sops/age/keys.txt
```

`age` and `sops` refuse to read a world-readable or group-readable key file. The check is deliberate: a key file that anyone on the system can read is a key file that anyone on the system can use.

8.9. `sops` opens the file but the save fails

The editor exited with a non-zero status, or the re-encryption failed. The most common cause is a syntax error in the edited YAML: SOPS parses the plaintext before re-encrypting, and a malformed file is rejected. The original `secrets.enc.yaml` is left unchanged; correct the YAML and try again.

8.10. The Fly deployment restarts repeatedly after `fly secrets set`

The new value is not one the application can use. Check the logs:

```
fly logs --app mywebapp
```

A `credentials.Load` error names the missing or invalid secret. Correct the value and run `fly secrets set` again, or roll back by re-setting the previous value.

9. Part 7 — What not to do

- **Do not commit `secrets.yaml`.** It is the plaintext form. The `.gitignore` catches it, but the pattern is a guard, not a guarantee. Verify with `git status --short` before every commit that touches secrets.
- **Do not decrypt to a file and re-encrypt.** Use `sops secrets.enc.yaml` and edit in place. The decrypt-to-file workflow leaves plaintext on disk.
- **Do not put secrets in `.env`.** The `.env` file holds configuration only. A secret in `.env` is a secret in two places, and the second place is not encrypted.
- **Do not put secrets in `fly.toml [env]`.** The `[env]` block is visible in the Fly dashboard and in the `fly.toml` file itself. Only `AGE_KEY` should be a Fly Secret, and only because it is the bootstrap key that decrypts everything else.
- **Do not reuse the age key across projects.** One key per project limits the blast radius of a compromise. If you have three projects, you have three age keys, each stored as the `AGE_KEY` secret of the corresponding Fly application.
- **Do not lose the age private key.** There is no recovery. If the key is gone, the ciphertext in the repository is unreadable and the secrets must be regenerated from scratch and re-encrypted to a new key. The application can survive this if the new secrets are written to a new SOPS file and deployed; it cannot survive losing the **values** if they are only in the SOPS file. Keep a backup.
- **Do not use a shared `AGE_KEY` across environments.** A staging deployment that can decrypt the production secrets is a staging deployment that can leak them. If you need staging, give it its own key and its own encrypted file.
- **Do not pass `AGE_KEY` on the command line in scripts.** Shell history, `ps` output, and CI logs all capture command lines. Use an env file, a CI secret, or `fly secrets set`.

10. Part 8 — Files referenced by this document

Path	Purpose
<code>secrets.enc.yaml</code>	The committed, encrypted secrets file. Edited with <code>sops secrets.enc.yaml</code> .
<code>secrets.yaml</code>	The plaintext working copy. Created by <code>sops</code> during editing, deleted on exit. Gitignored.
<code>~/.config/sops/age/keys.txt</code>	The age private key on a developer machine. Mode <code>0600</code> .
<code>internal/credentials/credentials.go</code>	The loader. Reads <code>AGE_KEY</code> , decrypts the SOPS file, exports the secrets into the environment.
<code>internal/credentials/parse.go</code>	The YAML parser used by the loader.
<code>.env</code>	Non-secret configuration. Gitignored. Copy of <code>.env.example</code> .
<code>.env.example</code>	The committed template for <code>.env</code> .
<code>.env.docker.local</code>	The untracked env file for the container. Contains <code>AGE_KEY</code> and the database host. Gitignored.
<code>fly.toml</code>	The Fly application manifest. Contains non-secret configuration in <code>[env]</code> . Does not contain secrets.
<code>AGE_KEY</code>	The Fly Secret holding the age private key. Set with <code>fly secrets set AGE_KEY=...</code> .

11. Appendix — Command reference

Command	Effect
<code>age-keygen -o FILE</code>	Generate a new age key pair, writing the private key to <code>FILE</code> and printing the public key to stderr.
<code>sops FILE</code>	Edit <code>FILE</code> in place. Decrypts to a temp file, opens <code>\$EDITOR</code> , re-encrypts on save.
<code>sops --decrypt FILE</code>	Decrypt to stdout. Leaves no file on disk.
<code>sops --encrypt FILE</code>	Encrypt to stdout. Does not modify <code>FILE</code> .
<code>sops --extract '["KEY"]'</code> <code>--decrypt FILE</code>	Decrypt a single value.
<code>sops --extract '["sops"]["age"]'</code> <code>--decrypt-keys FILE</code>	Show the recipients the file is encrypted to.
<code>sops --add-age RECIPIENT FILE</code>	Add a new age recipient. The file remains decryptable by all existing recipients as well.
<code>sops --rm-age RECIPIENT FILE</code>	Remove an age recipient. The file is no longer decryptable with that key.
<code>fly secrets set KEY=VALUE</code>	Set a secret. Triggers a rolling restart.
<code>fly secrets list</code>	List secret names and digests. Values are not retrievable.
<code>fly secrets unset KEY</code>	Remove a secret. Triggers a rolling restart.
<code>fly secrets import</code>	Read secrets from stdin in dotenv format.
<code>fly logs</code>	Stream the application's logs.
<code>fly deploy</code>	Build and deploy the current directory.

12. Appendix — Environment variables read by internal/credentials

Variable	Required	Meaning
AGE_KEY	In containers	The age private key, as a value. Read first. In development this is unset and the loader falls back to the key file.
SOPS_AGE_KEY	No	Set by the loader from AGE_KEY. The SOPS library reads this.
SOPS_AGE_KEY_FILE	No	A path to a file containing the age private key. Read if AGE_KEY is unset.
APP_ENV	In production	development, staging, or production. Governs the fail-closed behaviour: production and staging refuse to start on a missing or short secret; development warns.
XDG_CONFIG_HOME	No	Honoured when locating the default age key path. If unset, \$HOME/.config is used.

13. Appendix — The `secrets.enc.yaml` schema

The file is a single-level YAML mapping of string keys to string values. The loader rejects nested maps: a secret is a scalar.

```
DB_PASSWORD: string
TEST_DB_PASSWORD: string
JWT_SECRET: string      # at least 32 bytes in production
CSRF_SECRET: string     # at least 32 bytes, different from JWT_SECRET
R2_ACCESS_KEY_ID: string
R2_SECRET_ACCESS_KEY: string
```

The `knownSecrets` list in `internal/credentials/credentials.go` names the keys the loader exports into the environment. It is a closed set: a key in the file that is not on the list is silently ignored, and a key on the list that is not in the file is a hard error in production.

When you add a secret, edit the file **and** the list. The two must agree.

The `count` in the `credentials loaded count=N` log line is the number of keys the loader exported. If it does not match the number of keys in the file, the two have diverged. That log line is the only runtime signal of the divergence; there is no other warning.